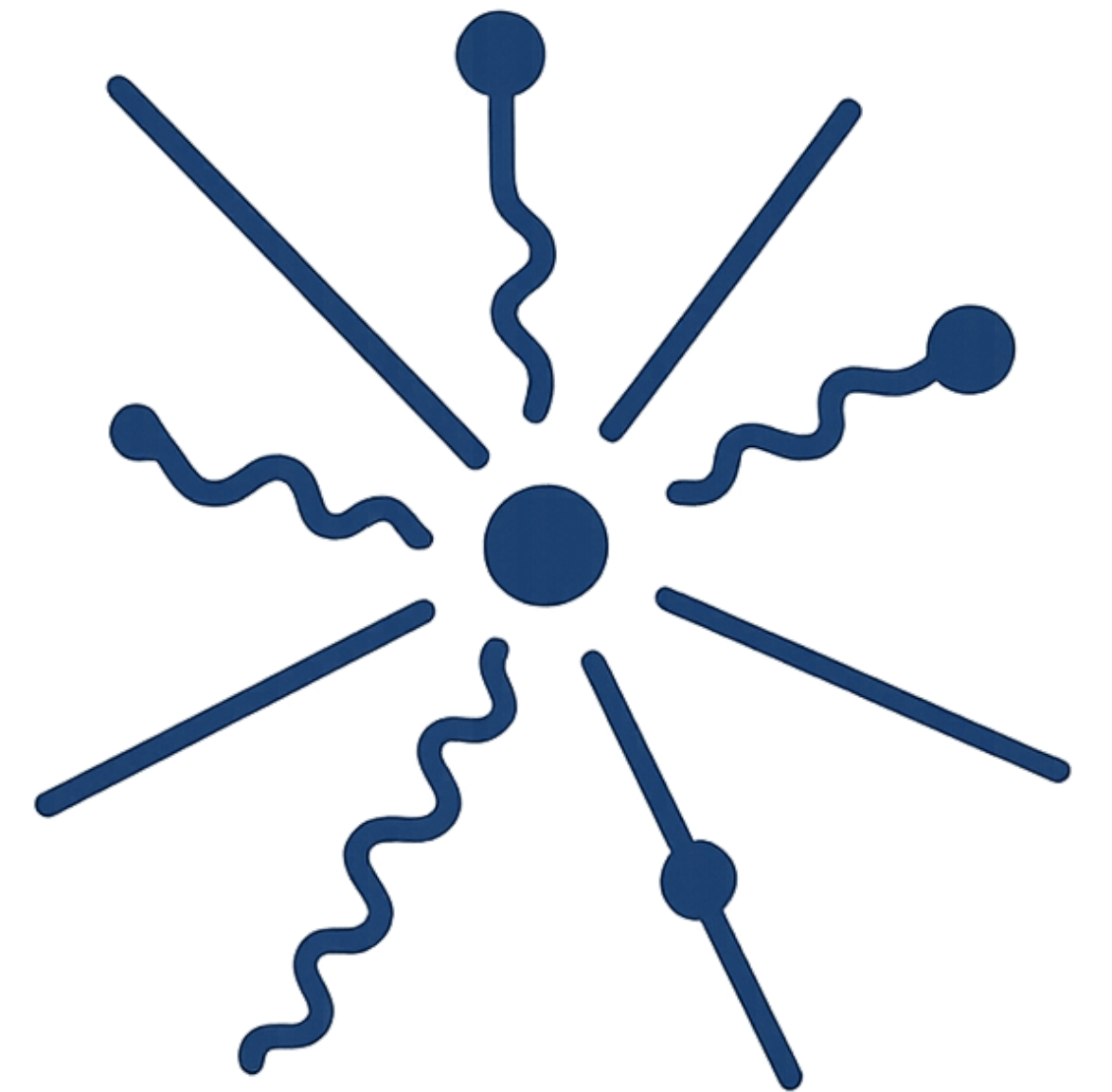
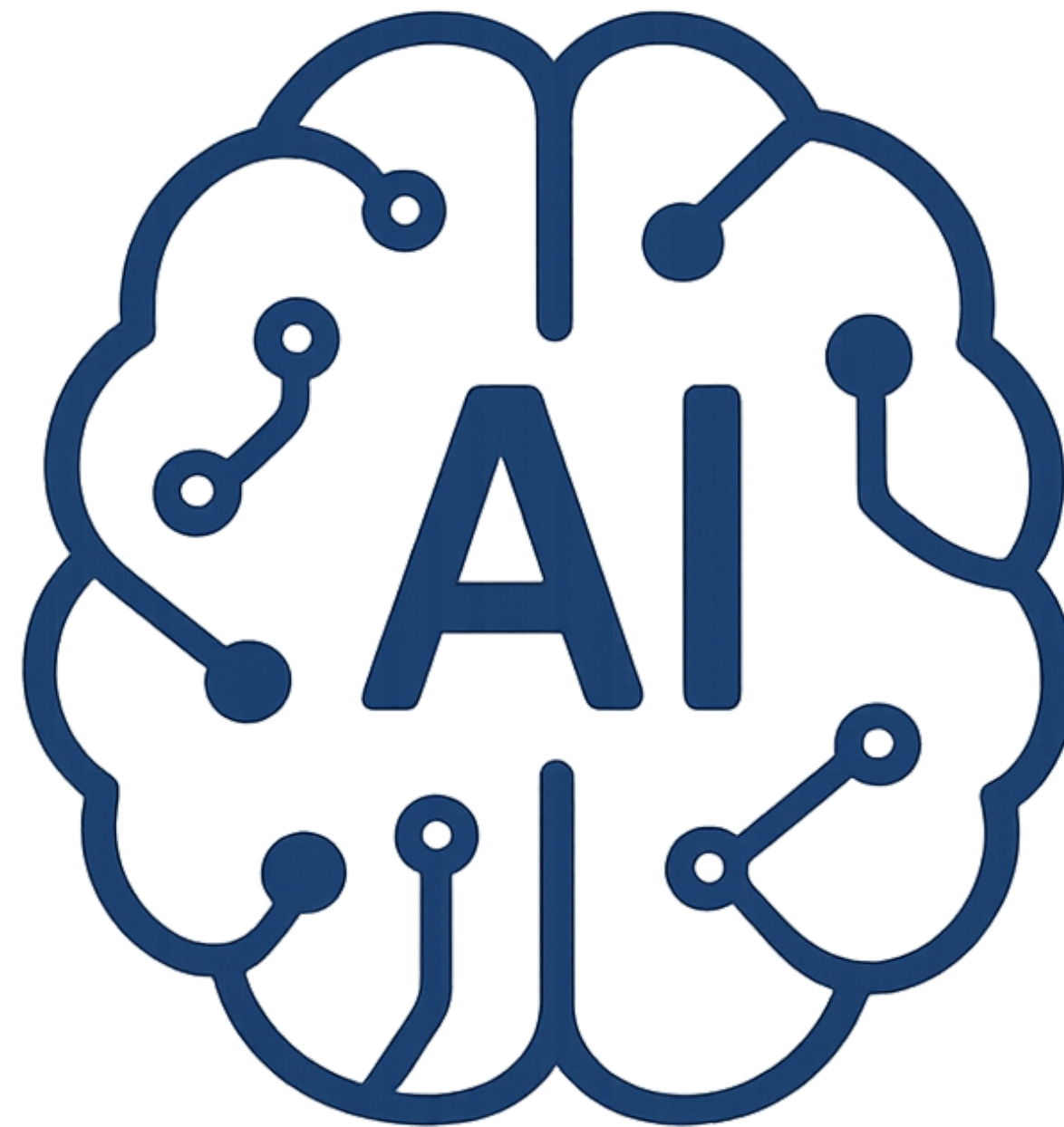
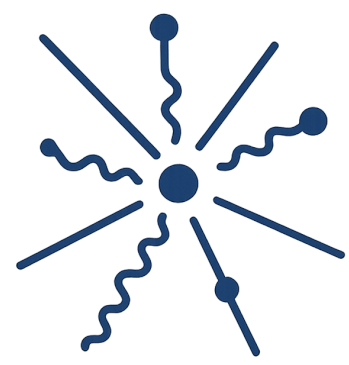
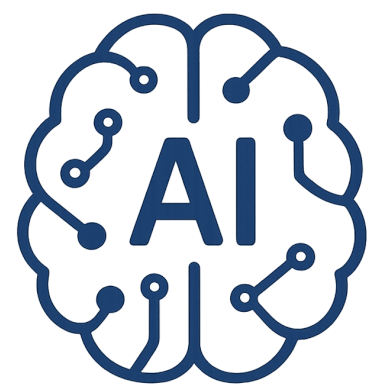


Introduction to

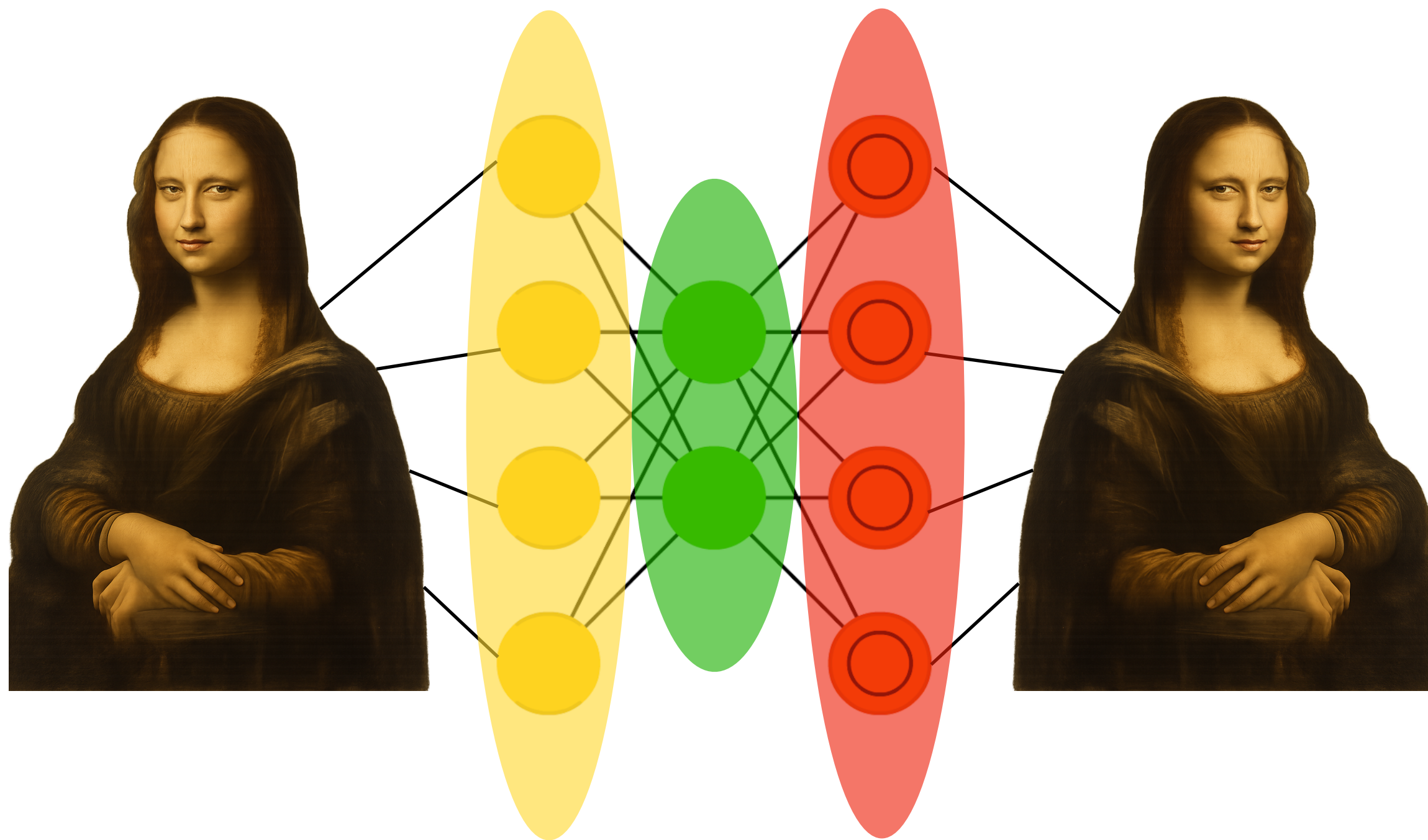
AI-Driven HEP

S. A. Fard - School of Physics (IPM)





AI-Driven HEP 21



Session 21

Latent Representation II

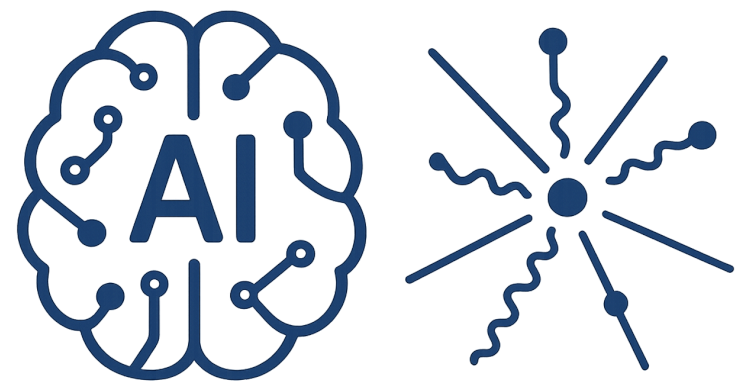


Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE, <https://arxiv.org/pdf/2010.11929v2>
- [3] Masked Autoencoders Are Scalable Vision Learners, <https://arxiv.org/abs/2111.06377v3>

<https://physics.ipm.ac.ir/~ansarifard/>





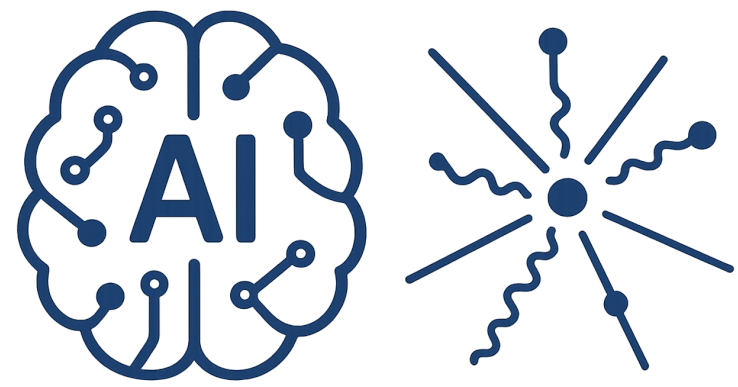
Generative Tasks



masked patch prediction

corrupt 50% of patch embeddings

replacing their embeddings with a learnable [mask]
embedding (80%), a random other patch embedding (10%)
or just keeping them as is (10%)



Generative Tasks



masked patch prediction

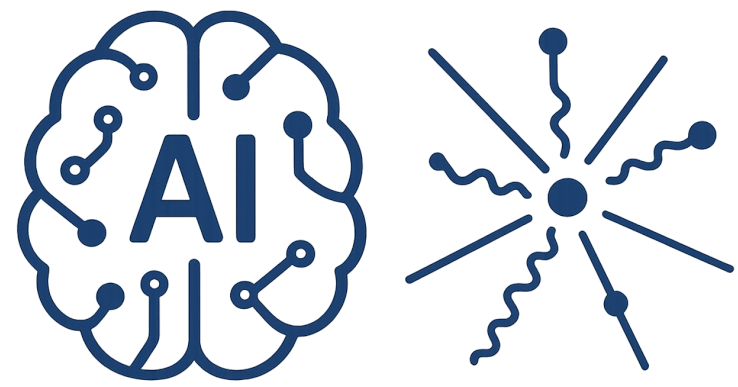
corrupt 50% of patch embeddings

replacing their embeddings with a learnable [mask]
embedding (80%), a random other patch embedding (10%)
or just keeping them as is (10%)

How to create a proper objective ?

Take the network output $z_i \in \mathbb{R}^D$

Linear classifier head $p(y_i | z_i) = \text{softmax}(Wz_i)$



Generative Tasks



masked patch prediction

corrupt 50% of patch embeddings

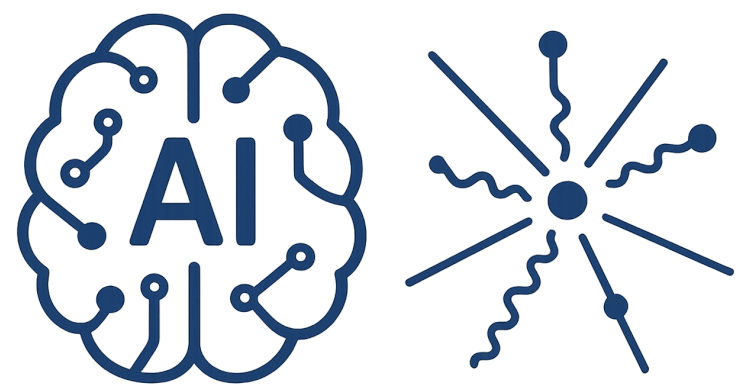
replacing their embeddings with a learnable [mask]
embedding (80%), a random other patch embedding (10%)
or just keeping them as is (10%)

How to create a proper objective ?

Take the network output $z_i \in \mathbb{R}^D$

Linear classifier head $p(y_i | z_i) = \text{softmax}(Wz_i)$

Number of classes ?



Generative Tasks



masked patch prediction

corrupt 50% of patch embeddings

replacing their embeddings with a learnable [mask]
embedding (80%), a random other patch embedding (10%)
or just keeping them as is (10%)

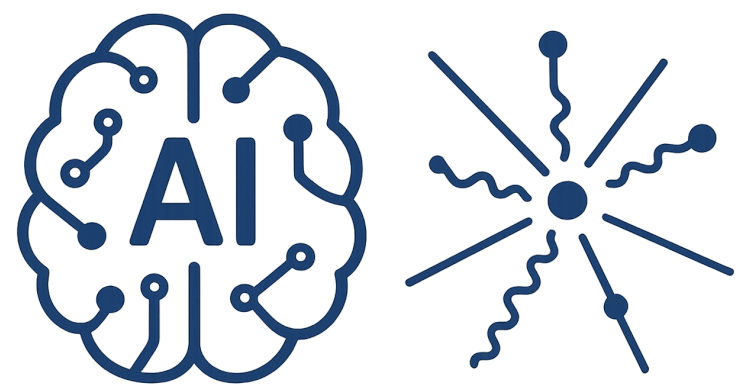
How to create a proper objective ?

Take the network output $z_i \in \mathbb{R}^D$

Linear classifier head $p(y_i | z_i) = \text{softmax}(Wz_i)$

Number of classes ?

$$16 \times 16 = 256 \quad \text{Dic : } 2^{256}$$



Generative Tasks



masked patch prediction

corrupt 50% of patch embeddings

replacing their embeddings with a learnable [mask]
embedding (80%), a random other patch embedding (10%)
or just keeping them as is (10%)

How to create a proper objective ?

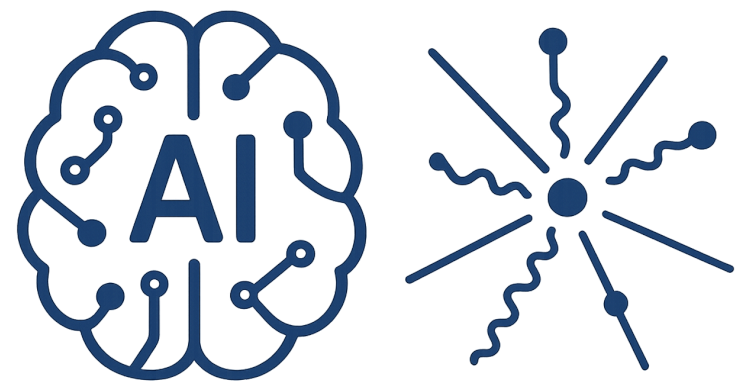
Take the network output $z_i \in \mathbb{R}^D$

Linear classifier head $p(y_i | z_i) = \text{softmax}(Wz_i)$

Number of classes ?

$$16 \times 16 = 256 \quad \text{Dic : } 2^{256}$$

Continuous Values
representation challenge



Generative Tasks



masked patch prediction

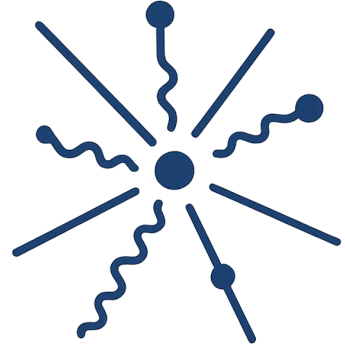
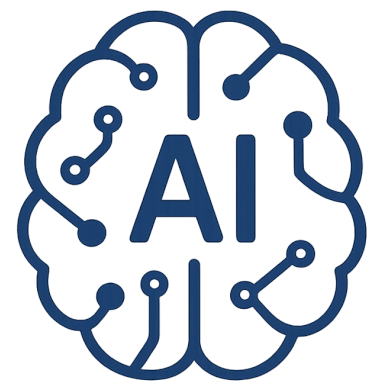
Each patch is summarized by its mean RGB color

3-bit mean color $(\bar{R}, \bar{G}, \bar{B})$

$2^3 = 8$ levels per channel

Total number of color classes

$$8 \times 8 \times 8 = 512$$



Autoregressive Transformer



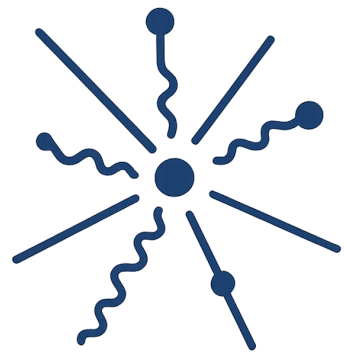
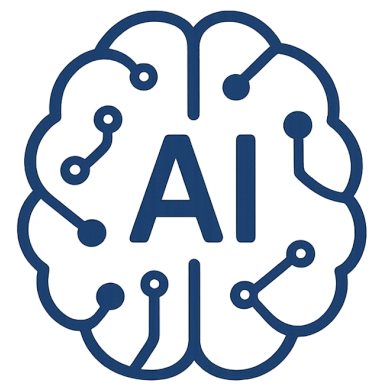
vector quantization



Consider a set of data vectors $x_1, \dots, x_N$ each of dimensionality D



Introduce a set of K codebook vectors $C = c_1, \dots, c_K$ also of dimensionality D



Autoregressive Transformer



vector quantization



Consider a set of data vectors $x_1, \dots, x_N$ each of dimensionality D

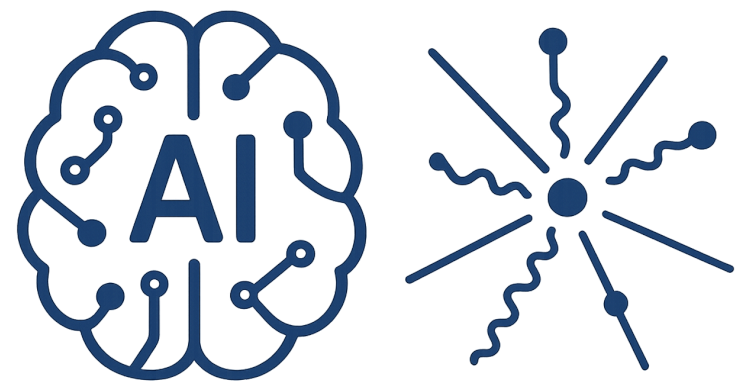


Introduce a set of K codebook vectors $C = c_1, \dots, c_K$ also of dimensionality D

$$K \ll D$$

Large K : accurate representation

Small K : greater compression



Autoregressive Transformer



vector quantization



Consider a set of data vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ each of dimensionality D



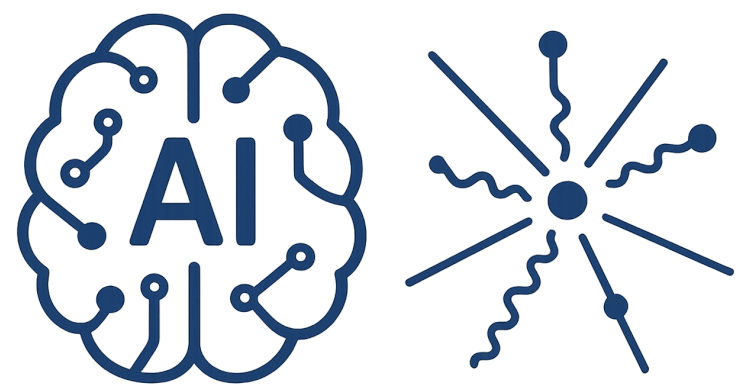
Introduce a set of K codebook vectors $C = c_1, \dots, c_K$ also of dimensionality D

$$K \ll D$$

$$\mathbf{x}_n \rightarrow \arg \min_{\mathbf{c}_k \in C} \|\mathbf{x}_n - \mathbf{c}_k\|^2$$

Large K : accurate representation

Small K : greater compression



Autoregressive Transformer



vector quantization

Consider a set of data vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ each of dimensionality D

Introduce a set of K codebook vectors $C = c_1, \dots, c_K$ also of dimensionality D

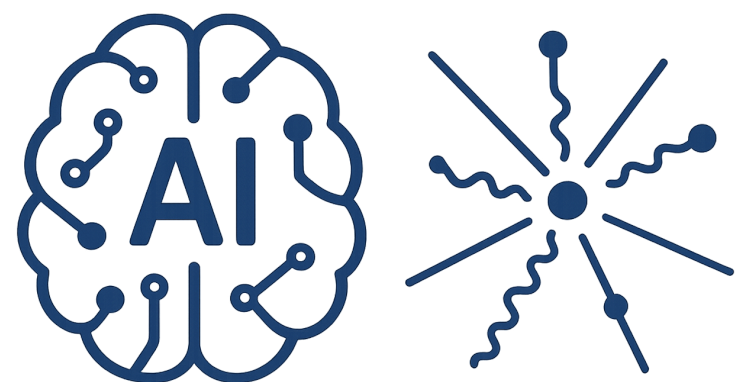
$$K \ll D$$

$$\mathbf{x}_n \rightarrow \arg \min_{\mathbf{c}_k \in C} \|\mathbf{x}_n - \mathbf{c}_k\|^2$$

Large K : accurate representation

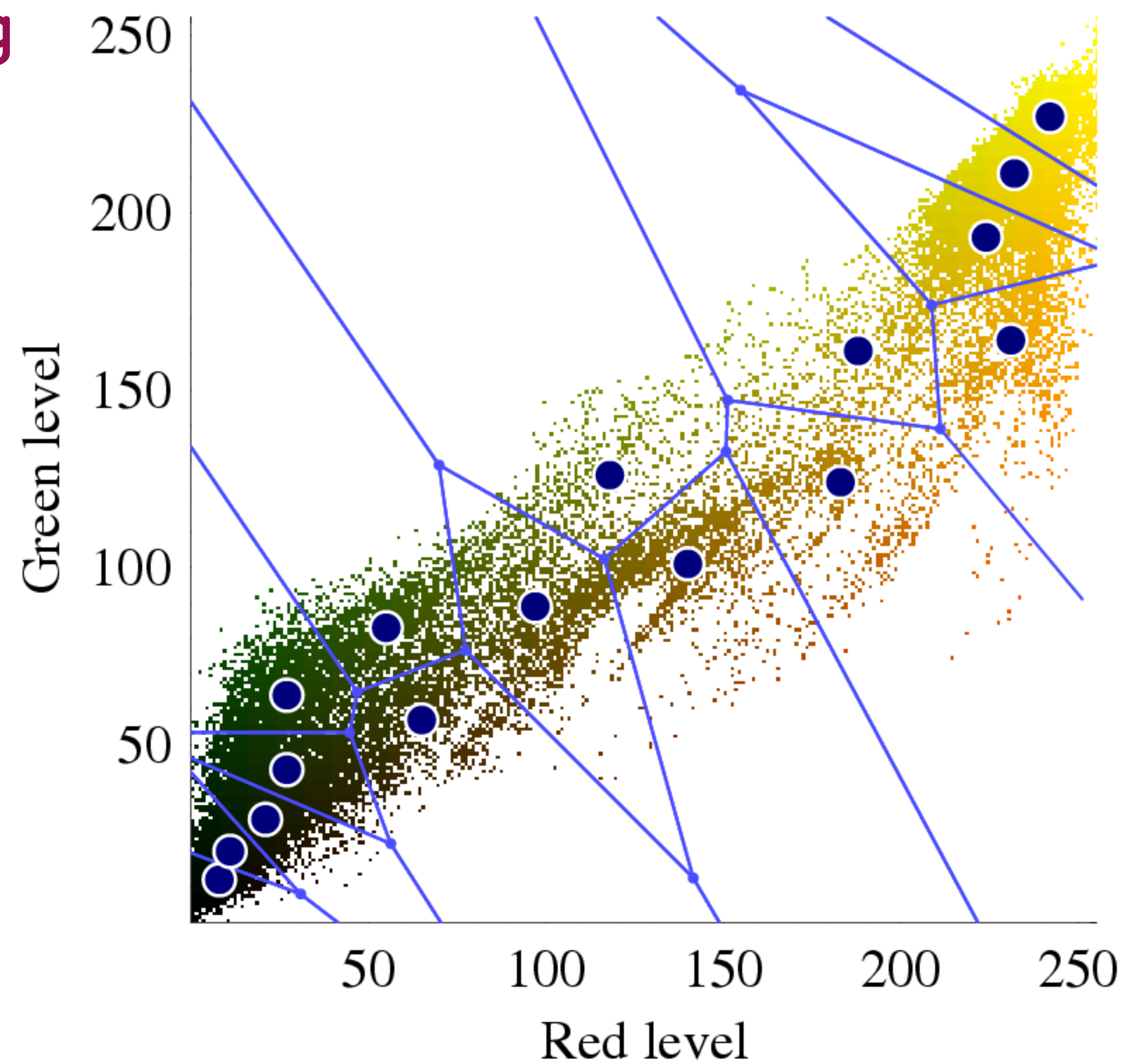
Small K : greater compression

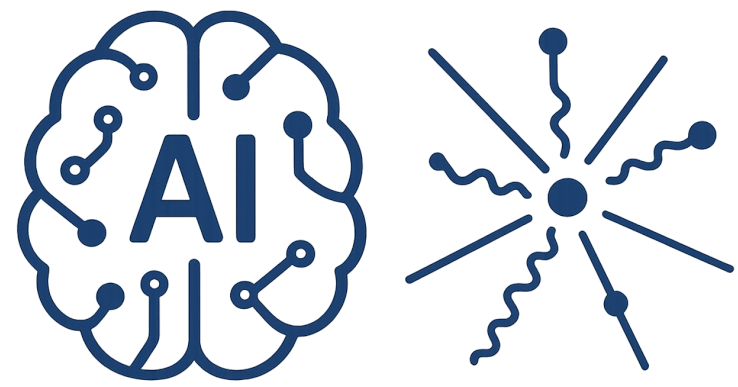
Model can then be trained to generate a sequence of codebook vectors



AI-Driven HEP 21

K-means Clustering





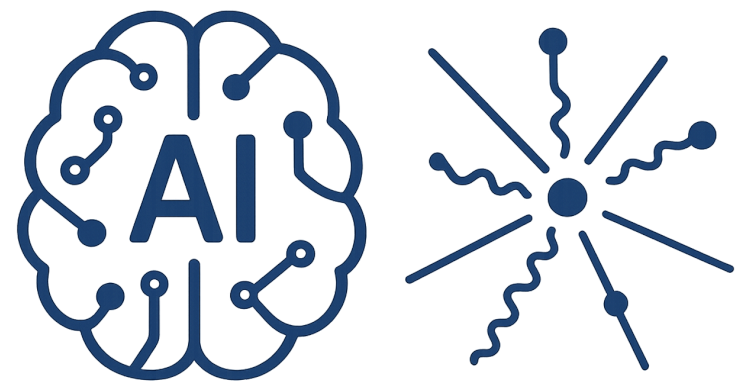
K-means Clustering

Expectation–Maximization (EM) algorithm

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_i - \mu_k\|^2$$

Expectation step

$$r_{nk} = \begin{cases} 1, & \text{if } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{otherwise.} \end{cases}$$



K-means Clustering

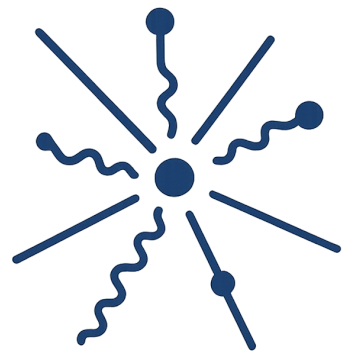
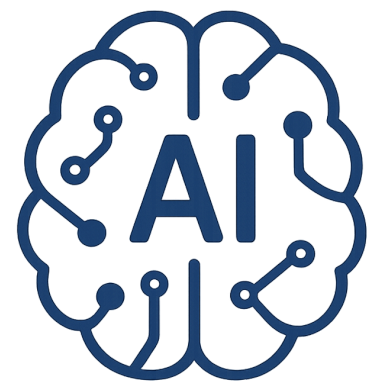
Expectation–Maximization (EM) algorithm

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_i - \mu_k\|^2$$

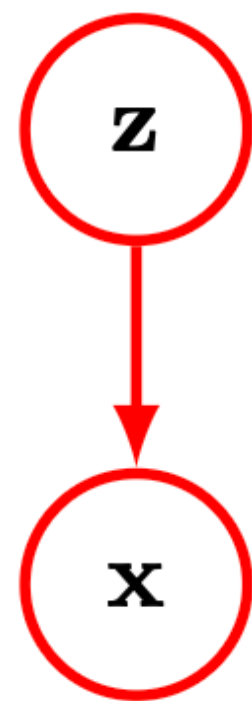
Maximization step

$$\frac{\partial J}{\partial \mu_k} = 0 \rightarrow 2 \sum_{n=1}^N r_{nk} \|x_i - \mu_k\| = 0$$

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$$

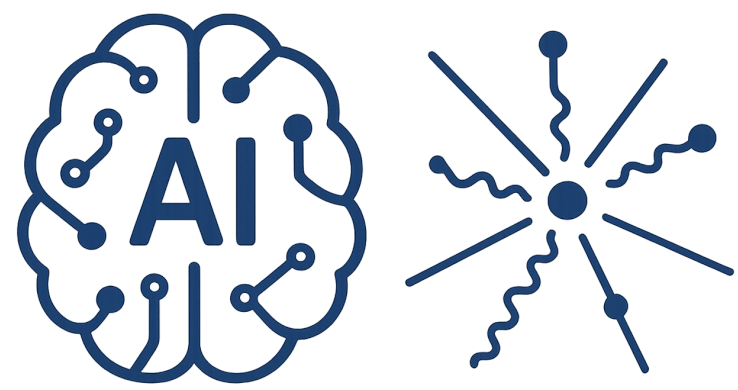


Mixtures of Gaussians

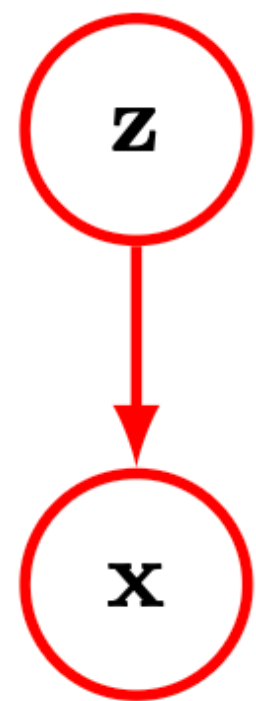


K-dimensional binary random variable z

$$p(z) = \prod_{k=1}^K \pi_k^{z_k} \quad p(z_k = 1) = \pi_k$$
$$\sum_k \pi_k = 1$$



Mixtures of Gaussians

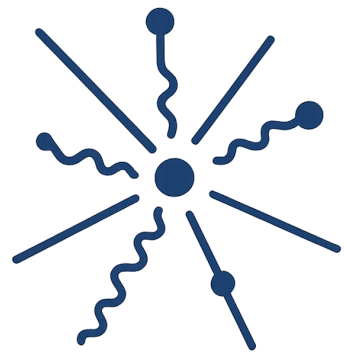
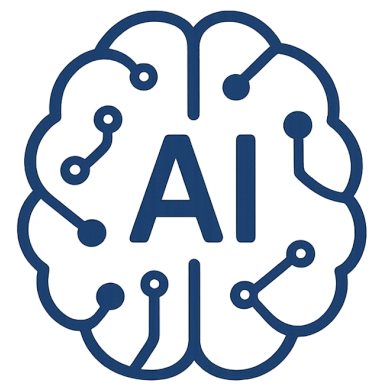


K-dimensional binary random variable z

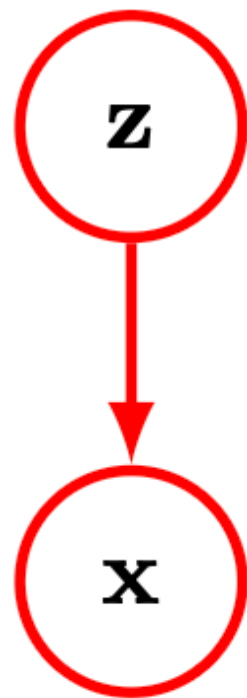
$$p(z) = \prod_{k=1}^K \pi_k^{z_k} \quad p(z_k = 1) = \pi_k$$
$$\sum_k \pi_k = 1$$

Conditional distribution of x given a particular value for z is a Gaussian

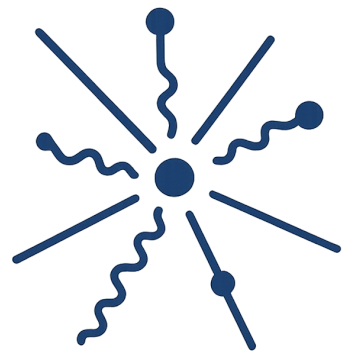
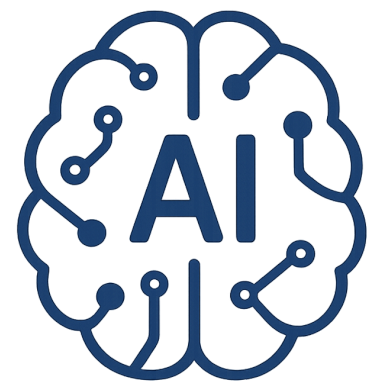
$$p(x | z) = \prod_{k=1}^K \mathcal{N}(x | \mu_k, \Sigma_k)^{z_k}$$



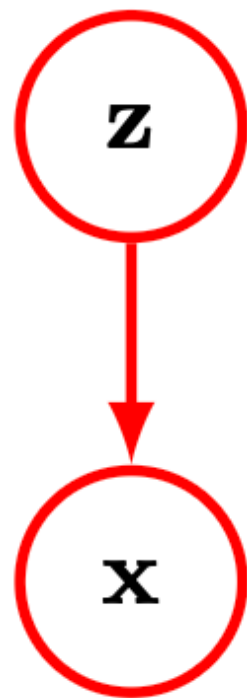
Mixtures of Gaussians



Quiz : Write down the marginal distribution of x , shown by $p(x)$



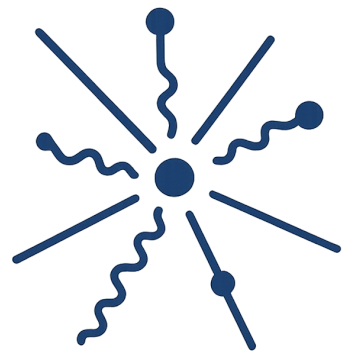
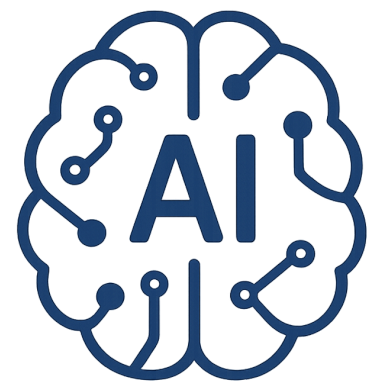
Mixtures of Gaussians



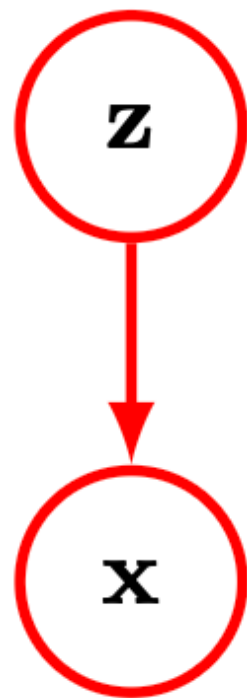
The marginal distribution of x

$$p(x) = \sum_z p(x, z) = \sum_k \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

For every observed data point x_n there is
a corresponding latent variable z_n



Mixtures of Gaussians



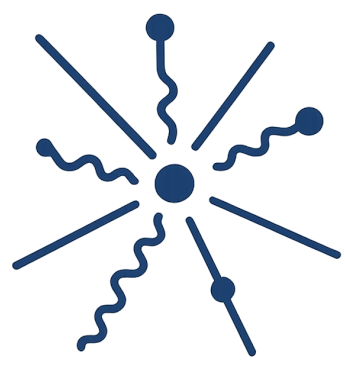
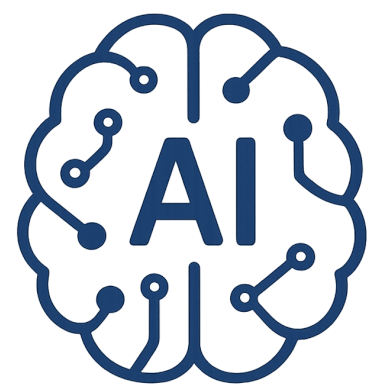
The marginal distribution of $\mathbf{x}$

$$p(x) = \sum_z p(x, z) = \sum_k \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

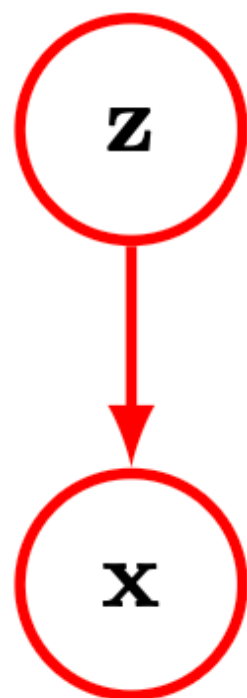
For every observed data point x_n there is
a corresponding latent variable z_n

Likelihood function

$$\ln p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$



Mixtures of Gaussians



The marginal distribution of $\mathbf{x}$

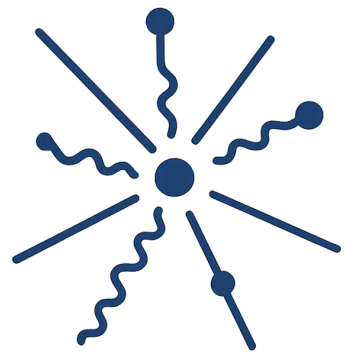
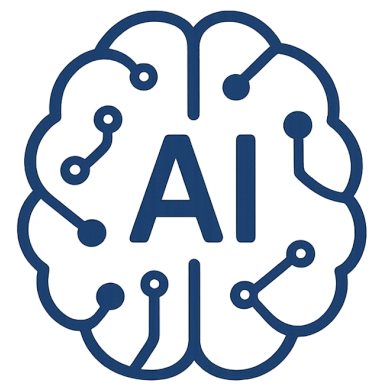
$$p(x) = \sum_z p(x, z) = \sum_k \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

For every observed data point x_n there is
a corresponding latent variable z_n

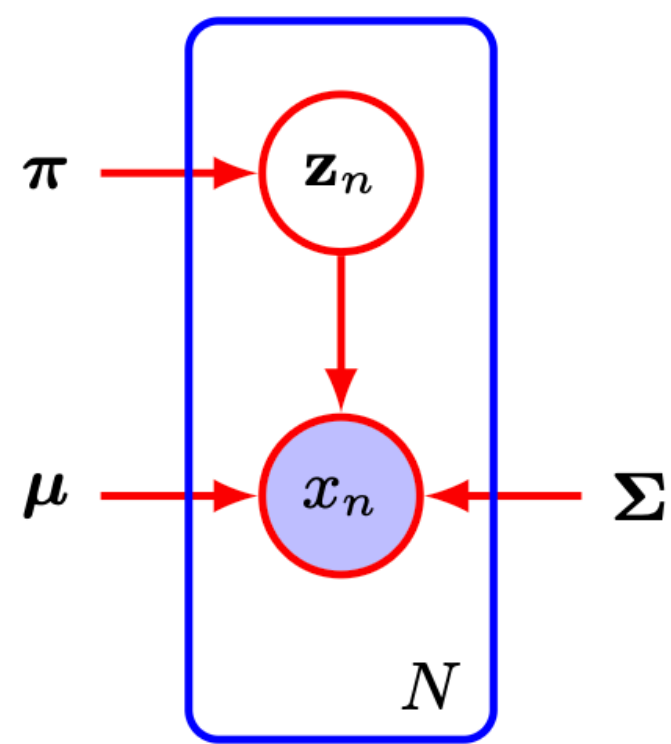
Likelihood function

$$\ln p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

$\mathbf{X} : (N \times D)$
 $\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$ unknown

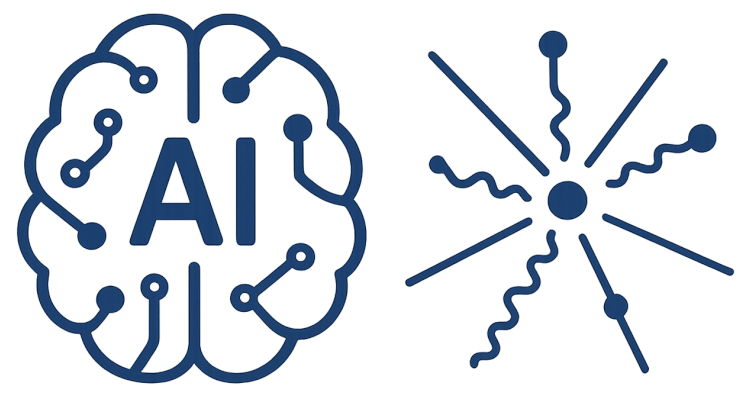


Mixtures of Gaussians

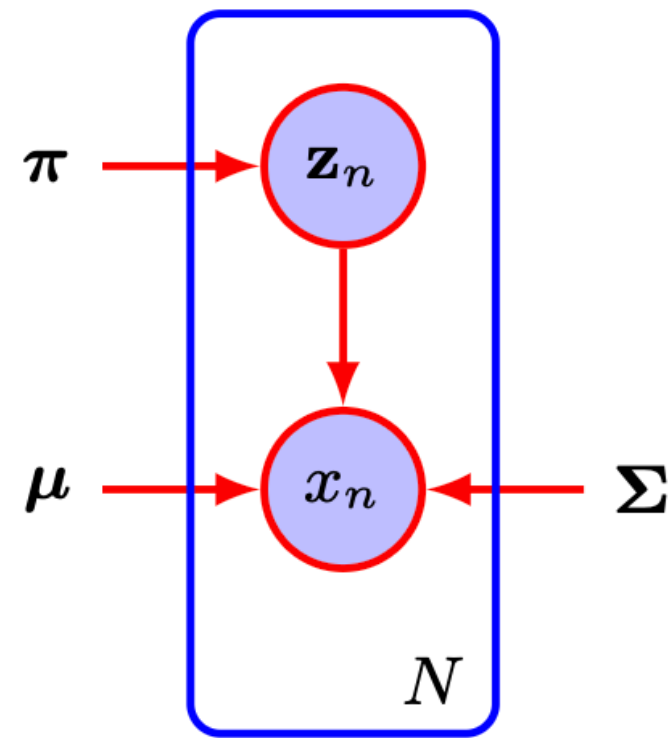


$$\ln p(\mathbf{X}|\boldsymbol{\theta}) = \ln \left\{ \sum_{\mathbf{z}} p(\mathbf{X}, \mathbf{z}|\boldsymbol{\theta}) \right\}$$

The presence of the sum makes the expression complicated
convergence challenges
Likelihood singularity



Mixtures of Gaussians



$$p(x, z) = p(z)p(x|z)$$

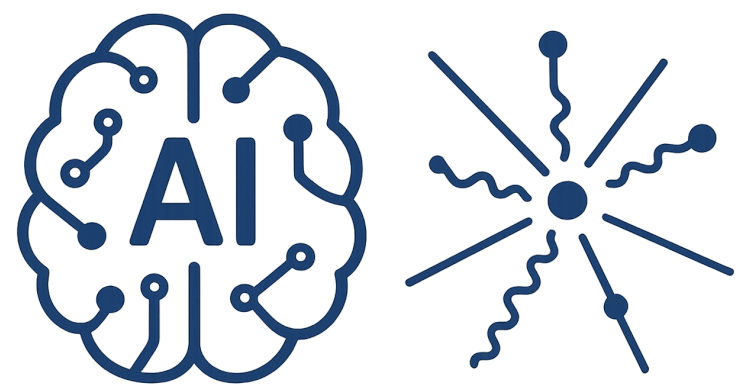
$$\ln p(\mathbf{X}, \mathbf{Z} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \{ \ln \pi_k + \ln \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \}$$

$$\mathbf{Z} : (N \times K)$$

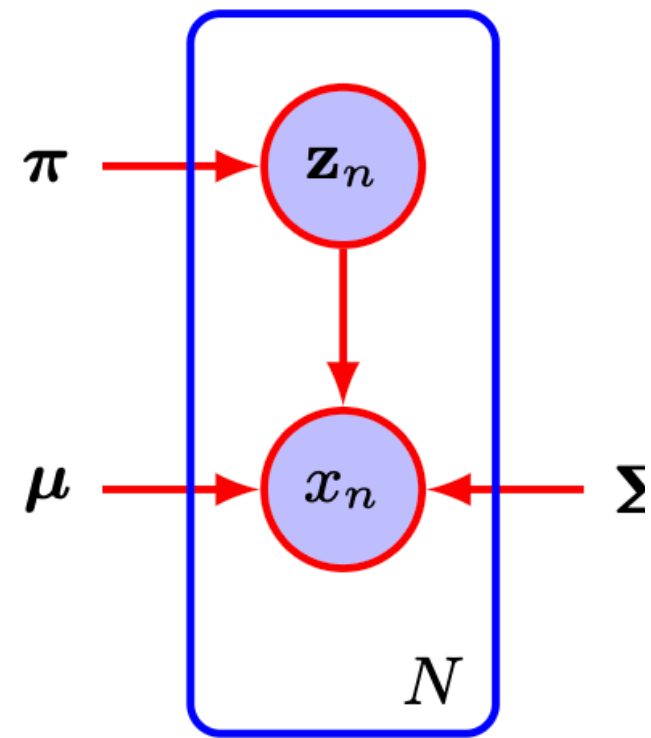
leads to a much simpler solution

i.i.d

However, z_{nk} is unknown

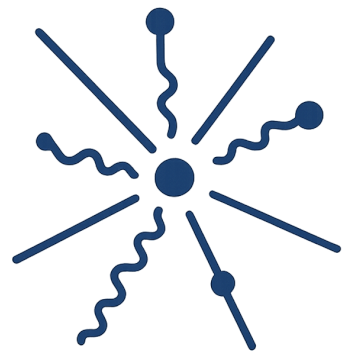
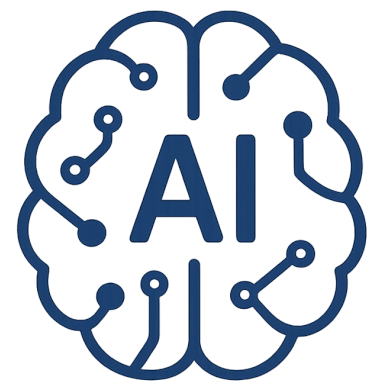


Mixtures of Gaussians

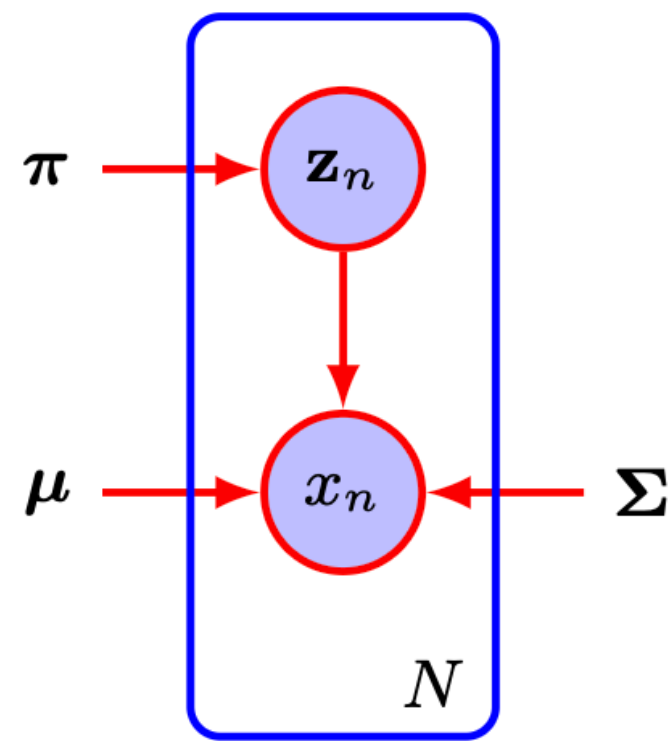


The expectation of z_{nk} with respect to the posterior distribution of the latent variables

$$\begin{aligned}
 \mathbb{E}[z_{nk}] &= \frac{\sum_{\mathbf{z}_n} z_{nk} \prod_{k'} [\pi_{k'} \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_{k'}, \boldsymbol{\Sigma}_{k'})]^{z_{nk'}}}{\sum_{\mathbf{z}_n} \prod_j [\pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)]^{z_{nj}}} \\
 &= \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} = \gamma(z_{nk}),
 \end{aligned}$$



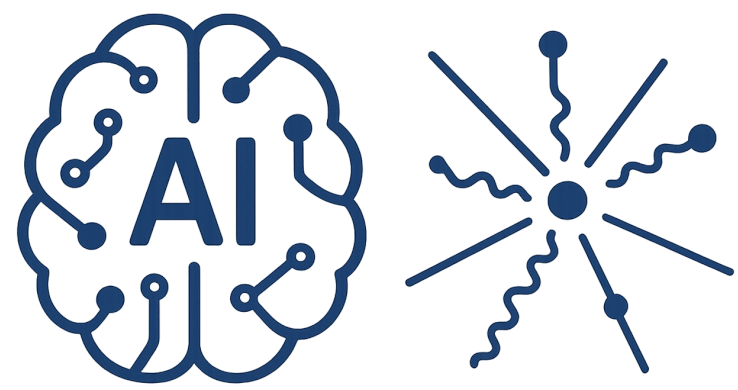
Mixtures of Gaussians



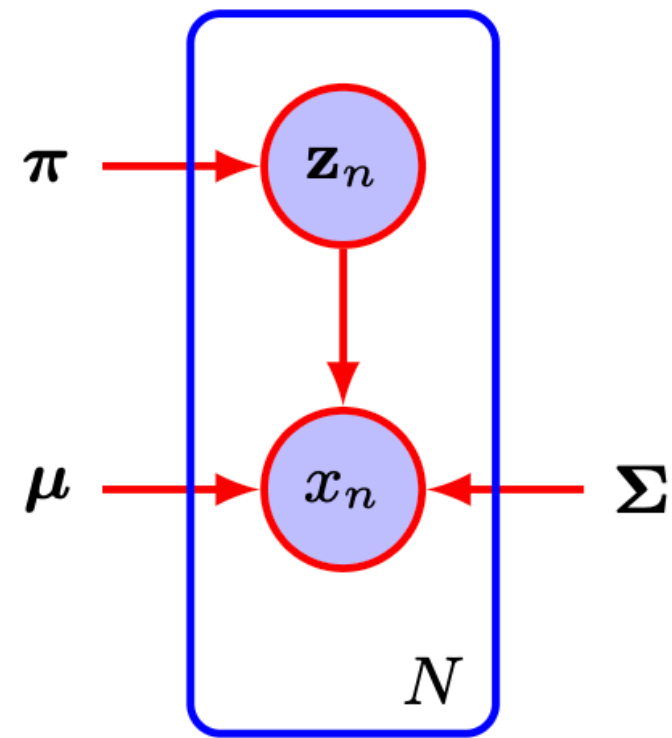
posterior responsibility

$$\gamma(z_{nk}) = p(z_n = k \mid x_n)$$

Latent representation linearizes learning



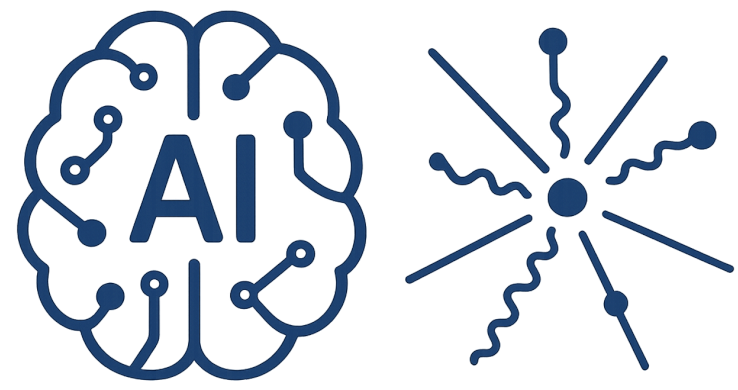
Mixtures of Gaussians



$$\mathbb{E}_{\mathbf{Z}}[\ln p(\mathbf{X}, \mathbf{Z} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})] = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) \{ \ln \pi_k + \ln \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \}$$

E step Initialize μ, π, Σ to evaluate $\gamma(z_{n,k})$

M step Keep $\gamma(z_{n,k})$ fixed and maximize likelihood



Comparison with K-means

In the limit of $\Sigma \rightarrow \epsilon \rightarrow 0$

$$\gamma(z_{nk}) = \frac{\pi_k \exp \{ -\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 / 2\epsilon \}}{\sum_j \pi_j \exp \{ -\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 / 2\epsilon \}} \quad \gamma(z_{n,k}) \rightarrow r_{n,k}$$

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n \quad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T$$

$$\pi_k = \frac{N_k}{N} \quad N_k = \sum_{n=1}^N \gamma(z_{nk})$$